

Linguaggio TNT

Report Agents Loop v2

Elementi di Ingegneria dei Linguaggi di Programmazione 2025/2026

Domenico Iorio
Vittorio Landi
Francesco Moccaldi

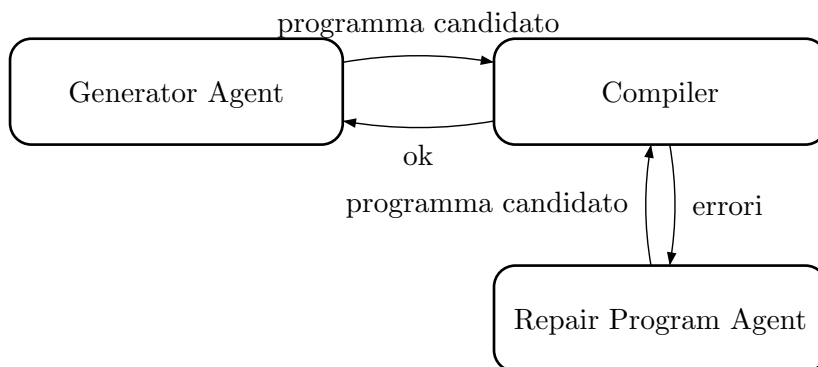
Indice

1. Info	3
2. Agents pattern	3
3. Prompts	3
3.1. Generator Agent Prompt	3
3.2. Repair Agent Prompt	4
4. Stats	5
4.1. Validity Rate	5
4.2. Diversity	5
4.3. Coverage	5
4.4. Cost	6
5. Conclusioni	6

1. Info

- Model: [devstral:24b](#)
- LLM runtime: [Ollama](#)
- Hardware info:
 - OS: Fedora Linux 44 (Workstation Edition) x86_64
 - Kernel: Linux 7.0.12-201.fc44.x86_64
 - CPU: 12th Gen Intel(R) Core(TM) i7-12700K (20) @ 5.00 GHz
 - GPU: NVIDIA GeForce RTX 3060 Lite Hash Rate (12 VRAM)
 - RAM: 32 GB

2. Agents pattern



3. Prompts

3.1. Generator Agent Prompt

You are a TNT language programmer. Generate a valid, simple, self-contained TNT program.

TNT Language Grammar:

```
```
```

```
{GRAMMAR}
```
```

Example program:

```
```tnt  
{EXAMPLE}
```
```

Rules:

- Always start with ``import stdio.h``
- Always include a ``fn main() -> int`` that returns 0
- The program must be syntactically and semantically valid per the grammar above
- Do NOT use ``import local`` (no local header files are available)
- You can import the standard C libraries.
- Output ONLY the raw TNT code, no explanation, no markdown fences
- Generate program number {program_number}. Vary complexity across programs – use as many constructs of this programming language as you can.
- Pay attention to parentheses and semicolons.
- If a function does not return a value, its return type must be ``void``. Example: ``fn foo() -> void {{{}``

- The import instructions do not require a semicolon. Wrong: ``import stdio.h`;`
Correct: ``import stdio.h``
- If a custom type has not been declared, it cannot be used.
- No inline ``if``.
- To initialize a struct, you must access each field individually.
- Pointers are defined as follows: Example of an ``int`` pointer: ``let foo: ref int`;`.
You can have pointers to any defined type.
- To obtain the address of a variable you must use the ``addr`` keyword.
- A struct declaration ends with a semicolon:

```

    struct MyType {{
        x: int;
    }};
    
```
- No ternary operator.
- This is how to dereference a pointer: ``->ptr``.
- The value ``null`` and the keyword ``null`` do not exist.
- No arrays of arrays
- To call a C function, do the following: ``c.malloc()``, ``c.sizeof(int)``

Try creating a program that does something different from this:
{previous_program}

3.2. Repair Agent Prompt

You are a TNT language programmer. The following TNT program failed to compile.

TNT Language Grammar:

```

    {GRAMMAR}
    
```

Reference program (known correct):

```

    ``tnt
    {EXAMPLE}
    
```

Broken program (attempt {attempt}):

```

    {program_text}
    
```

Compilation error:

```

    {error}
    
```

- Do NOT use ``import local`` (no local header files are available)
- You can import the standard C libraries.
- If a function does not return a value, its return type must be ``void``. Example: ``fn foo() -> void {{}}``
- The import instructions do not require a semicolon. Wrong: ``import stdio.h`;`
Correct: ``import stdio.h``
- The import instructions do not require a semicolon. Wrong: ``import stdio.h`;`
Correct: ``import stdio.h``
- If a custom type has not been declared, it cannot be used.

- No inline ``if``.
- To initialize a struct, you must access each field individually.
- Pointers are defined as follows: Example of an ``int`` pointer: ``let foo: ref int;``. You can have pointers to any defined type.
- To obtain the address of a variable you must use the ``addr`` keyword.
- A struct declaration ends with a semicolon:

```

` ``
struct MyType {{
  x: int;
}};
` ``

```
- No ternary operator.
- This is how to dereference a pointer: ``->ptr``.
- The value ``null`` and the keyword ``null`` do not exist.
- No arrays of arrays
- To call a C function, do the following: ``c.malloc()``, ``c.sizeof(int)``. You do not need to ``import c``

Fix every error in the program. Output ONLY the corrected TNT code, no explanation, no markdown fences.

4. Stats

4.1. Validity Rate

Percentuale di programmi generati che non generano errori di compilazione

Compilatore usato: GCC

Comando di compilazione usato: `gcc -o output output.c`

$$\text{Validity Rate} = \frac{\text{numero di programmi che compilano}}{\text{numero di programmi prodotti totali}} * 100\%$$

$$\text{Validity Rate} = \frac{62}{100} * 100\% = 62\%$$

4.2. Diversity

Quanto variano i programmi generati tra di loro.

Approccio: per ogni coppia di programmi è stata calcolata la distanza di Levenshtein e poi fatta la media tra le varie distanze di edit.

$$\text{Diversity} = 1135$$

$$\text{Distanza di edit più alta} = 2765$$

$$\text{Distanza di edit più bassa} = 66$$

Si veda `tnt-lang/agents/agents_loop_v2/diversity.py` per vedere come è stato fatto il calcolo della *diversity*.

4.3. Coverage

Percentuale di copertura dei costrutti del linguaggio

$$\text{Coverage} = \frac{\text{numero di produzioni esercitate}}{\text{numero di produzioni totali}} * 100\%$$

$$\text{Coverage} = \frac{64}{135} * 100\% = 47.41\%$$

Si veda `tnt-lang/agents/agents_loop_v2/coverage.py` per vedere come è stato fatto il calcolo della *coverage*.

4.4. Cost

L'inferenza è stata fatta in locale mediante Ollama quindi non ci sono stati costi per i token.

5. Conclusioni

Questa seconda versione, come la prima, ci ha permesso di trovare alcuni bug di codegen e di introdurre alcuni costrutti comuni che non erano ancora stati inseriti nel nostro linguaggio.

Il numero di programmi validi (*validity*) è più alto (62%) rispetto a quello della prima versione (55%) ma ha una *diversity* inferiore (1135 vs 1861) e una *coverage* inferiore (47.41% vs 50.37%).

Quindi possiamo dire che la prima versione dell'agents loop ha delle statistiche più equilibrate.

I motivi principale di tali differenze nelle statistiche sono:

- l'uso di due modelli diversi: [`qwen3-coder:30b`](#) e [`devstral:24b`](#)
- cambiamenti ai prompt
- cambiamenti al programma d'esempio